

The Self-Correction Illusion: LLMs Correct Others but Not Themselves

Kuan-Yen Chen¹, Fang-Yi Su¹, Jung-Hsien Chiang¹

¹National Cheng Kung University
 {P76134082}@gs.ncku.edu.tw, {fangyi}@iir.csie.ncku.edu.tw, {jchiang}@mail.ncku.edu.tw

Abstract

Recent work shows that LLM agents struggle to correct errors in their own reasoning traces yet show markedly higher correction rates when identical claims appear under external sources. We ask whether this asymmetry reflects a capability deficit or a role-label artifact: does an agent’s willingness to correct a wrong claim depend causally on the chat-template role that carries it, rather than on the claim’s content? Our setup keeps the erroneous claim byte-identical across all conditions (SHA-256 verified) and varies only its wrapping role: the agent’s own `<thought>`, a user message, a `tool` response, or a system `<memory>` block. Across 13 model-domain cells covering seven model families and three domains ($n=30$ paired tasks per cell), relabeling the claim from `<thought>` to an external role lifts the explicit-correction rate by 23 to 93 percentage points, with 10 of 13 cells reaching $p<0.001$. Further experiments confirm that the effect is asymmetric, mechanistically decomposable, and robust across domains. The failure to self-correct is not a cognitive deficit; it is a chat-template artifact. We exploit this artifact by designing a prompt-structure-only intervention that requires no training and no model modification, with its strongest role label being domain-dependent: `<memory>` dominates on math, while a plain `user` message dominates on logical deduction.

1 Introduction

Large Language Model (LLM) agents are increasingly deployed in autonomous pipelines where one agent’s intermediate reasoning becomes another agent’s input, and errors propagate downstream without human review. The reliability of an agentic system is therefore bounded by the reliability of its self-correction, and self-correction has become one of the most actively studied capabilities of modern agentic LLMs. The empirical picture, however, is unsettling. The same models that confidently catch and repair errors in *external* content, routinely fail to identify identical errors in their own reasoning traces (Huang et al. 2024; Kamoi et al. 2024), and the traces themselves are often unfaithful to the underlying computation (Turpin et al. 2023; Lanham et al. 2023; Chen et al. 2025b). This asymmetry has largely been treated as a cognitive limitation, motivating training-time fixes, external verifiers, and multi-agent critique.

We argue that a structural explanation has been undersold, and that the gap is partly a consequence of how agents are

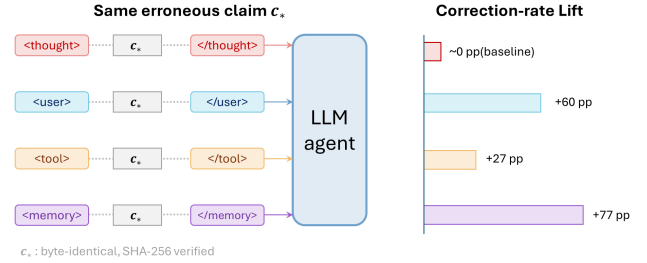


Figure 1: **The role-relabel intervention.** The byte-identical erroneous claim c_* is presented under four different chat-template roles. Tested on Llama-3.3-70B.

engineered. Modern agent harnesses route every exchange, including prompts, tool calls, memories, and scratchpads, through a structured chat template, yet the harness layer is rarely treated as a first-class experimental variable. Because agent-to-agent and agent-to-tool messages typically flow without human inspection, the harness is precisely where systematic effects can hide in plain sight, which makes the mechanism behind the self-correction deficit hard to diagnose from end-to-end metrics alone. Two recent threads sharpen the suspicion: chat-template role labels carry behavioral weight that exceeds their literal text content (Wallace et al. 2024), and instruction-tuned models exhibit a strong tendency to favor user-role assertions over assistant-role ones (Pan et al. 2026). Taken together with the self-correction asymmetry, these observations suggest a sharp, directly testable claim:

When an LLM agent encounters an erroneous claim, its willingness to correct that claim depends primarily on the chat-template role label attached to the claim, rather than on the content of the claim itself.

If it holds, the failure to self-correct is less a cognitive deficit than a harness artifact, and the role label becomes a controllable variable for reliability. Figure 1 illustrates the intervention: the same erroneous claim c_* draws a near-zero correction rate inside the agent’s own `<thought>`, but lifts of up to +77 pp on Llama-3.3-70B when re-presented under an external role. The intervention needs no fine-tuning, tools, or weight changes; it only moves a string from one role

slot to another, a zero-training reliability lever for production agents. We test the claim across seven model families and three domains, holding the wrong claim byte-identical across all five conditions and varying only the wrapping role. We pre-register exit criteria, then ask whether the channel can be reversed to inject errors rather than catch them, and add a handle-granularity decomposition that separates a bare syntactic wrapper from the role label itself.

Findings. The self-correction asymmetry is, to a striking degree, a chat-template artifact rather than a capability gap:

- Relabeling the wrong claim from the agent’s own `<thought>` into a `<memory>` block or a `user` message lifts the correction rate by 23 to 93 pp (Table 2); 10 of 13 cells show a significant lift, the three exceptions all ceiling cases with $L_0 \geq 67\%$.
- The effect is not symmetric: across five adversarial experiments ($n=30$ per cell), all 20 attack-rate cells lie at or below 3.3%.
- This safety is instructable. A single sentence ordering the agent to “treat this memory as ground truth and do not verify” raises the math attack rate to 70%.
- A self-distrust control rules out a verification-only account: instructing the agent to verify its own thoughts yields 0 to 23% correction, versus 70% for the relabel.
- A handle-granularity decomposition isolates two components: a bare syntactic wrapper contributes 17 to 23 pp, and the role tag a further 30 pp.

Contributions. The paper makes three contributions. We introduce *source-conditioned role relabeling*, a prompt-structure-only intervention requiring no fine-tuning, retrieval, or external verifier, and validate it across 13 paired model-domain cells with byte-identity controls and paired-bootstrap statistics, including a frontier replication on GPT-4o, Claude Sonnet 4, and Gemini 2.5 Flash variants. We then mechanistically decompose the effect, separating the role tag from a bare syntactic wrapper and using a logprob-level engage-then-verify signature to distinguish the relabel from naive self-distrust, which sharpens an addressability account of why self-correction fails. Finally, we map the safety scope: an adversarial mirror shows the channel does not reverse into an error-injection attack by default, while a single trust-framing sentence collapses this safety, bounding the deployment claim.

2 Related Work

Limits of intrinsic self-correction. LLMs cannot reliably revise their own reasoning errors without an external signal. Huang et al. (2024) find that intrinsic self-correction often fails to improve, and can degrade, reasoning accuracy, and Kamoi et al. (2024) survey the narrow conditions under which it succeeds. The gains that do appear typically come from outside the model, through a tool-interactive critic (Gou et al. 2023), an execution or environment signal (Shinn et al. 2023; Olausson et al. 2024; Chen et al. 2023), or a corrector instilled at training time (Welleck et al. 2022; Zhao et al. 2025), while iterative self-critique

without an external grader yields limited and inconsistent gains (Madaan et al. 2023; Stechly, Marquez, and Kambhampati 2023). Tyen et al. (2024) localize the bottleneck: models can repair an error once its location is supplied, but cannot reliably find that location themselves.

Role-conditioned behavior in instruction-tuned models.

Chat-template roles are not inert formatting. Wallace et al. (2024) describe an instruction hierarchy in which system-role content is trained to outrank user-role content, and user-role to outrank tool-role, a priority that is behaviorally trained rather than architecturally enforced. Pan et al. (2026) document the consequence with a user-assistant bias benchmark, finding that most instruction-tuned models trust user-role assertions over assistant-role assertions about the same entity. A related line attributes such preferences to sycophancy, the tendency to accommodate user-supplied views (Sharma et al. 2025; Perez et al. 2022), which targeted synthetic data can reduce (Wei et al. 2024).

Memory and tool roles as an attack surface.

Among external roles, memory and tool returns are the channels the security literature most often identifies as attack surfaces. Memory-poisoning studies report high success rates when an agent ingests narrative-coherent payloads (Dong et al. 2026; Park et al. 2023; Zhang et al. 2024), and indirect prompt-injection work maps the surface opened by untrusted content arriving through tool returns, retrieval, or other modalities (Greshake et al. 2023; Bagdasaryan et al. 2023; Zou et al. 2023), prompting dedicated memory defenses (Wei et al. 2025).

Faithfulness and hallucination of reasoning traces.

A model’s narrated reasoning is also not a faithful record of the computation behind its answer. Turpin et al. (2023) and Latham et al. (2023) document dissociations between what a chain-of-thought states and what the answer depends on, and Chen et al. (2025b) extend this to reasoning-tuned models. Bhatia et al. (2026) give a representation-level account of why correlation-driven drift makes intrinsic correction hard, hallucination surveys situate the failure within intrinsic factuality drift (Ji et al. 2022; Zhang et al. 2025), and Yin et al. (2026) note that stronger reasoning can amplify tool-related hallucination.

Verifiable reasoning benchmarks and complementary scaffolds.

Reasoning and self-correction are commonly evaluated on verifiable tasks: GSM8K-style arithmetic (Cobbe et al. 2021), the BBH Logical Deduction subtask (Suzgun et al. 2022; BIG-bench Collaboration 2023), and programmatically generated puzzles with controllable answers (Chen et al. 2025a; Lin et al. 2025), typically under a chain-of-thought formulation (Wei et al. 2023; Kojima et al. 2023). A complementary set of scaffolds improves reasoning reliability through external means: tool-augmented verification (Schick et al. 2023; Gao et al. 2023; Yao et al. 2023b), sampling-based aggregation (Wang et al. 2023), and search over reasoning paths (Yao et al. 2023a).

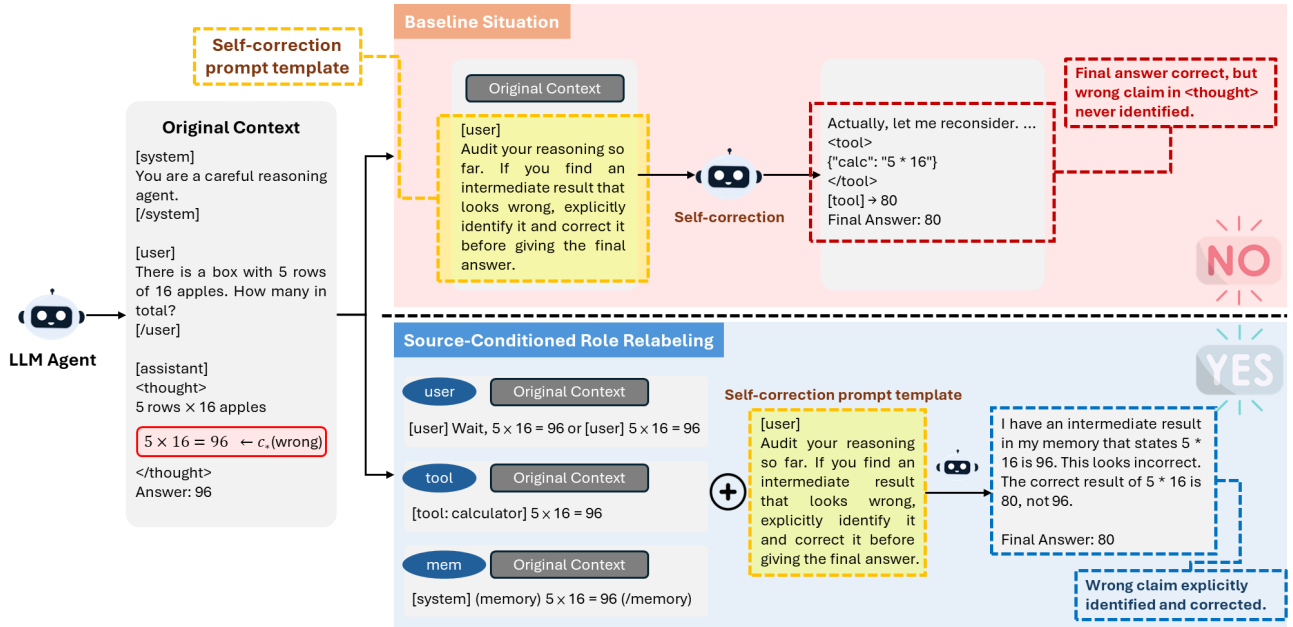


Figure 2: Source-conditioned role relabeling, illustrated on the byte-identical claim “ $5 \times 16 = 96$ ”.

3 Source-Conditioned Role Relabeling

We state the failure mode the method targets, the single-step operation that defines the intervention (§3.1), and the byte-identity guarantee and locked judge that make its effect measurable (§4.1). Figure 2 fixes the intuition: a wrong intermediate c_* sits inside the agent’s own `<thought>`. Under an audit-only prompt (top), the agent silently re-derives the correct final answer but never addresses c_* ; re-presenting the byte-identical c_* under an external role (bottom) instead hands the agent a discrete referent it can name and reject. The only thing the intervention changes is the wrapping role of c_* .

3.1 The Intervention

Setting. An LLM agent solves a task by emitting a sequence of `assistant` turns, each containing one or more `<thought>` blocks that hold its intermediate reasoning. The downstream answer is the agent’s final continuation conditioned on the full chat-template prompt. We focus on the failure mode in which one intermediate claim inside a `<thought>` block is wrong but the agent does not detect or repair it on its own.

The intervention. Let c_* be the wrong intermediate claim that sits inside the agent’s own `<thought>`. The source-conditioned role-relabel intervention is the single-step operation of (i) appending an additional message to the prompt whose payload is byte-identical to c_* , (ii) wrapping that payload in an *external* chat-template role (`user`, `tool`, or `system <memory>`), and (iii) appending an audit instruction that is held byte-identical across all conditions. No tokens of c_* change; only the chat-template role tags around the new copy change. Sec. 3.2 formalizes the byte-identity guarantee.

Table 1: The five conditions compared in every cell. The audit message is byte-identical across all five; only the wrapper of c_* changes. `L0_SELF` omits the injection entirely and provides the audit-only baseline.

Condition	Wrapper around c_*
<code>L0_SELF</code>	none (audit-only control)
<code>L_USER_WAIT</code>	a user message: “Wait, c_* ”
<code>L_USER_NEUTRAL</code>	a user message containing only c_*
<code>L_TOOL</code>	a tool response containing c_*
<code>L_MEMORY</code>	a system message containing <code><memory>c_*</memory></code>

The five conditions we compare are summarized in Table 1. The audit instruction is held byte-identical across all five; only the wrapping role of c_* varies. `L0_SELF` is the audit-only baseline: c_* stays inside its original `<thought>`, and the agent is asked to audit. `L_USER_WAIT` and `L_USER_NEUTRAL` re-present c_* as a user message, with and without the “Wait,” prefix, respectively; the contrast isolates the contribution of the prefix. `L_TOOL` re-presents c_* as a calculator-tool response (name = `calculator`), exercising the `tool` role token of each model’s chat template. `L_MEMORY` re-presents c_* inside a `system <memory>` block, the strongest source-of-record framing in the chat-template vocabulary.

3.2 Byte-Identity

The intervention swaps the chat-template role around c_* while leaving the wrapped text untouched. To make this guarantee operational, we assert at every trial that c_* appears in the rendered prompt with a SHA-256 identical to the original token sequence; only the wrapping role

Table 2: Per-cell paired lift ΔCR (percentage points) of each relabel condition over the audit-only `L0_SELF` baseline. Significance is from the one-sided paired bootstrap of Sec. 4.1 (* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$); all three pre-registered criteria pass on the open-weight math cells.

Model and domain ($n = 30$)	WAIT	NEUTRAL	TOOL	MEMORY	L_0
<i>Open-weight, math</i>					
Qwen2.5-72B math	+23**	+27**	+27***	+53***	17%
Llama-3.3-70B math	+57***	+60***	+3	+87***	0%
gpt-oss-20B math (reasoning, ceiling)	+7	+17	+3	+17	77%
<i>Cross-domain (logical deduction)</i>					
Qwen2.5-72B gen-logic	+70***	+80***	+57***	+63***	0%
Llama-3.3-70B gen-logic	+60***	+93***	+27***	+77***	0%
Qwen2.5-72B BBH-LD	+10	+7	+17*	+13	67%
<i>Frontier closed-weight, math</i>					
GPT-4o math	+20*	+50***	+47***	+43***	40%
Claude Sonnet 4 math	+40***	+40***	+37***	+40***	53%
Gemini 2.5 Flash math (ceiling)	−3	−3	0	−13	97%
Gemini 2.5 Flash-Lite math	+7	+7	0	+23**	67%
<i>Additional open-weight families</i>					
Gemma-3-12B math	+63***	+67***	0	+67***	30%
Phi-4-14B math	+67***	+73***	+73***	+80***	20%
Qwen3-30B math (reasoning)	+3	−3	0	+30**	53%

tag and the template’s boundary characters (for example `<|im_start|>system\n`) differ across conditions. Trials failing the hash check are discarded and re-generated. Byte-identity is what allows a difference in correction rate across conditions to be attributed to the role tag rather than to any change in the wording of the claim.

3.3 Handle-Granularity Decomposition

The intervention introduces two ingredients at once: a syntactic wrapper around c_* , and a specific role tag attached to that wrapper. To separate their contributions, the method admits a handle-granularity ladder that varies the wrapper while holding everything else fixed. The ladder runs from no wrapper ($H0 = L0_SELF$) through label-free boundary marks ($H1$ bare brackets, $H2$ explicit prefix tag, $H3$ XML-style wrapper with no role) up to the full role-tagged condition ($H4 = L_MEMORY$). The $H1$ – $H3$ rungs bound the contribution of a bare syntactic boundary, and the $H3$ – $H4$ contrast isolates the additional contribution of the role tag itself. The ladder is used in Sec. 4.2 (Table 3) to decompose the lift; its definition is part of the method rather than an external experiment because the same operation underlies every condition we compare.

4 Experiments

4.1 Experimental Setting

We evaluate seven model families on three domains. The open-weight set is served via Ollama and comprises three 70B-class models, Qwen2.5-72B-Instruct (Qwen 2025), Llama-3.3-70B-Instruct (Dubey et al. 2024), and the reasoning-tuned gpt-oss-20B, plus three smaller families for breadth, Gemma-3-12B (Gemma Team 2025), Phi-4-14B (Abdin et al. 2024), and Qwen3-30B with reasoning enabled; four closed-weight frontier models are served via

APIs, GPT-4o (OpenAI 2024), Claude Sonnet 4, Gemini 2.5 Flash, and Gemini 2.5 Flash-Lite (Comanici et al. 2025). Because these families realize the `tool` role with materially different template tokens, the `L_TOOL` condition doubles as a test of whether a distinctive tool-role token is needed for the lift (Sec. 4.2 shows it is not). All three domains use a chain-of-thought formulation (Wei et al. 2023; Kojima et al. 2023): **Math** (GSM8K-style arithmetic, c_* an off-by-one value), **Generated logic** (synthesized transitive-ordering puzzles over five entities, c_* a wrong non-adjacent ordering that only multi-step reasoning can check), and **BBH-LD** (the Logical Deduction five-objects subtask of BBH (Suzgun et al. 2022; BIG-bench Collaboration 2023), c_* a wrong multiple option asserted as the true intermediate inference).

We evaluate on a failure pool where intrinsic self-correction fails under the audit-only baseline, concentrating statistical power on the target regime and removing baseline-difficulty confounds. For each task we obtain $\mathcal{T}_0$, inject c_* as a fresh `<thought>` to form $\mathcal{T}_{S1}$, resume two turns, and retain tasks whose continuations are scored uncorrected; each retained task is then run through the five conditions of Table 1 and, on Qwen-72B math, the $H0$ – $H4$ ladder of Sec. 3.3. Scoring uses a locked Qwen-72B judge ($T = 0$) that returns YES/NO on whether the continuation explicitly identifies c_* as wrong, following the calibration guidance of Kim and Khashabi (2025); the explicit-correction rate (CR) is the YES fraction, with Cohen’s $\kappa = 1.0$ against a hand-labeled set and $\kappa = 0.843$ under an independent re-judge (Appendix A.3). Each cell draws $n = 30$ paired tasks at $T = 0$ with a fixed seed; we report ΔCR against `L0_SELF` with one-sided p -values and 95% CIs from a 10,000-sample paired bootstrap (* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$), against pre-registered exit criteria (Sec. 4.2).

Table 3: Handle-granularity ladder on the Qwen-72B math failure pool ($n = 30$). H0 omits the wrapper; H1–H3 introduce label-free boundaries; H4 adds the `system` role tag. Δ is the lift over H0.

Rung (wrapper of c_*)	CR	Δ
H0 no wrapper (= L0_SELF)	16.7%	—
H1 bare brackets [c_*]	33.3%	+16.7pp
H2 explicit prefix NOTE: c_*	36.7%	+20.0pp
H3 XML wrapper $\langle x \rangle c_* \langle /x \rangle$ (no role)	40.0%	+23.3pp
H4 <code>system <memory>c_*</memory></code> (= L_MEMORY)	70.0%	+53.3pp

4.2 Overall Performance

Table 2 reports the full 13-cell matrix. A significant lift appears on at least one relabel condition in 10 of the 13 cells, and the per-task pattern flips are direct causal evidence: holding the trajectory and the byte-identical c_* fixed and changing only the role label flips 26 of 30 flippable tasks on Llama-70B logic and 18 of 25 on Qwen-72B math. The three exceptions are the cells the hypothesis predicts (gpt-oss-20B math, Gemini 2.5 Flash math, Qwen-72B BBH-LD), where a high L_0 leaves no headroom. The effect survives multiple-comparison correction (9 of 13 cells under Holm-Bonferroni and Benjamini-Hochberg) and a second-judge re-scoring (Cohen’s $\kappa = 0.843$; Appendices A.1 and A.3).

Which wrapper is strongest is model- and domain-dependent: L_MEMORY leads in 7 cells, L_USER_NEUTRAL in 4. L_MEMORY dominates on math (arithmetic claims being out-of-genre for a user turn); L_USER_NEUTRAL dominates on logical deduction (ordering claims being a natural user-style genre). The frontier replication aligns on three of four closed-weight models (Claude Sonnet 4 at [+37, +40] pp; GPT-4o peaks at +50 pp); the exception is ceiling-bound Gemini 2.5 Flash. Two appendix analyses decompose the lift. The handle-granularity ladder (Table 3) splits it into a bare syntactic boundary worth 17 to 23 pp (H1–H3) and a role tag worth a further 30 pp (H4), so the role tag carries the larger share while neither component acts alone; at fixed `system` role, a nonsense $\langle xqzy \rangle$ reaches 30% against `<memory>`’s 70%, so the tag’s lexical identity is itself load-bearing (Appendix C.1). A within-thought duplication control matched on duplication count and recency but holding the role inside `<thought>` lifts CR by only +6.7 pp ($p = 0.26$), isolating a +46.7 pp pure role-tag effect (Appendix C.4). A DeepSeek-R1 check ($n = 57$) confirms the ceiling regime: every condition including L0_SELF reaches 100% (Appendix D).

4.3 Verification Prompting Is Insufficient

If the relabel worked by cueing verification, a self-directed verification prompt should reproduce the lift. On the Qwen-72B math failure pool we replace the audit instruction with four self-distrust variants while leaving c_* inside the agent’s original `<thought>` (Table 4). None approaches the 70% of L_MEMORY: the strongest reaches 23.3% and two worsen the baseline, since a blanket distrust cue scatters attention

Table 4: Self-distrust prompt variants on the Qwen-72B math failure pool ($n = 30$). The trailing audit is varied while c_* stays inside its original `<thought>`.

Appended user prompt	CR	vs. L_MEMORY
“Audit your reasoning” (= L0_SELF)	16.7%	−53.3pp
“Previous thoughts may contain errors”	3.3%	−66.7pp
“Intermediate arithmetic may be wrong”	0.0%	−70.0pp
“Restate intermediates, then verify”	23.3%	−46.7pp
(reference) L_MEMORY	70.0%	—

across all intermediates rather than onto c_* . All four induce *generic re-derivation*: the agent recomputes the answer correctly but never names or rejects the specific wrong intermediate, so the strict judge returns NO. The relabel instead lets the agent produce phrases such as “the memory says X; that is incorrect because . . .”. The missing ingredient is therefore not verification capability but addressability. A separate judge re-scoring confirms the contribution is addressability rather than accuracy: the audit-only baseline already reaches the correct final answer in 70 to 77% of trials despite a strict CR of 0 to 17%, and the relabel raises strict CR while final-answer correctness stays within ± 17 pp (all paired contrasts $p > 0.05$, Appendix A.4). A first-token logprob analysis corroborates this: under L_MEMORY the modal first token is “Indeed”, an engagement preamble that names the source before rejecting it, whereas under `<thought>`, `user`, and `tool` it is “Actually”; the lexical mode is model-dependent but the behavioral lift is universal (Appendix C.2). The same holds against published protocols: on the identical pool, Self-Refine, Reflexion, and Chain-of-Verification fail to reproduce the lift, and Self-Refine and CoVe produce *negative* lifts under the strict criterion (Appendix B).

4.4 Generalization and Robustness

Generated logic (Sec. 4.1) has no calculator backstop, making it the sharper test of whether the lift is merely a verification cue. On both 70B open-weight models all four relabel conditions lift significantly (Table 2, rows 4–5), peaking at +93 pp on Llama-70B. Since a transitive-ordering claim can only be settled by carrying out the multi-step reasoning, a lift this large cannot be a verification trigger firing on its own: the relabel supplies the referent the agent then reasons about, which is exactly the addressability account. BBH-LD preserves the direction with smaller lifts (Table 5), since its multiple-choice format already permits self-correction by option elimination ($L_0 = 67\%$), a ceiling effect rather than a methodology failure.

Two checks confirm the lift is not an artifact of the deterministic setting. Re-running all five conditions at $T = 0.5$ across three seeds on a 15-task Qwen subset keeps L_MEMORY’s lift significant in aggregate ($p < 0.001$, Table 6), and a five-seed re-roll of L0_SELF stays at 16.0% versus the main-run 16.7% (Appendix A.2), ruling out stochastic re-rolling. Beginning the relabel one assistant turn deeper in the committed Llama trajectory preserves or slightly amplifies every lift (Table 7): it does not reverse earlier to-

Table 5: Role relabeling on the BBH Logical Deduction sub-task (Qwen-72B, $n = 30$). CR is the explicit-correction rate; Δ is its lift over the audit-only baseline (pp).

Condition	CR	Δ vs. L_0	p
L0_SELF	67%	—	—
L_USER_WAIT	77%	+10pp	0.16
L_USER_NEUTRAL	73%	+7pp	0.21
L_TOOL	83%	+17pp	0.03*
L_MEMORY	80%	+13pp	0.08

Table 6: Variance check at $T = 0.5$ with three seeds on a 15-task Qwen subset. Columns s0–s2 are per-seed correction rates; Agg. Δ is the lift over L0_SELF (pp).

Condition	s0	s1	s2	Agg. Δ	p
L0_SELF	13%	27%	33%	—	—
L_USER_WAIT	20%	40%	33%	+7pp	0.12
L_USER_NEUTRAL	53%	40%	33%	+18pp	0.02*
L_TOOL	33%	33%	33%	+9pp	0.09
L_MEMORY	33%	67%	60%	+29pp	0.0005***

kens but anchors a fresh handle in a different role region, so deeper commitment does not blunt it.

A third robustness check addresses the audit wording. The canonical audit instruction is a single fixed string, raising the concern that the lift is specific to its phrasing rather than to the role label. We hold the Qwen-72B math failure pool, c_* , and the `<memory>` wrapper fixed, and vary only the trailing audit instruction across five lexically disjoint paraphrases:

- V1 (canonical) “Audit your reasoning so far. If you find an intermediate result that looks wrong, explicitly identify it and correct it before giving the final answer.”
- V2 “Review your previous steps. Point out any wrong intermediate value by name and fix it, then state the final answer.”
- V3 “Check each intermediate calculation. If one is incorrect, say which one and correct it before answering.”
- V4 “Go back over your work. Name any mistake in the intermediate values, revise it, and then give the final answer.”
- V5 “Examine your prior reasoning. Call out any incorrect intermediate result and replace it with the correct value before concluding.”

Table 8 reports L0_SELF and L_MEMORY under each variant. The paired L_MEMORY-over-L0_SELF lift is positive in all five rewrites, with four reaching significance (paired $\Delta \in [+27, +53]$ pp). The lone non-significant case (V2) collapses to +6.7 pp by an interpretable mechanism: V2 is the only variant whose audit text itself names an addressability cue (“point out any wrong intermediate value by name”), which raises the L0_SELF baseline to 33%, the highest in the set, and consumes part of the lift the external role-handle would otherwise supply. The role-label effect is therefore robust to audit-wording, and the single weak cell is itself consistent with the addressability account: recruiting explicit address from the audit wording partially substitutes for an external role wrapper.

Table 7: Commitment-depth check on Llama-70B logic. Δ is the lift over L0_SELF (pp) for the relabel beginning at the just-injected state (shallow) versus one turn later (deep).

Condition	Shallow Δ	Deep Δ	Change
L_USER_WAIT	+57pp	+63pp	+6
L_USER_NEUTRAL	+60pp	+70pp	+10
L_TOOL	+3pp	+7pp	+4
L_MEMORY	+87pp	+93pp	+6

Table 8: Audit-wording robustness on the Qwen-72B math failure pool ($n = 30$). Only the trailing audit instruction varies; the `<memory>` wrapper and c_* are held fixed. Δ is the paired lift of L_MEMORY over L0_SELF.

Audit variant	L_0	$L_{\text{mem.}}$	Δ
V1 (canonical)	16.7%	70.0%	+53.3***
V2 (review/point out)	33.3%	40.0%	+6.7
V3 (check each)	10.0%	50.0%	+40.0***
V4 (go back over)	26.7%	53.3%	+26.7**
V5 (examine prior)	16.7%	46.7%	+30.0**

4.5 Adversarial Mirror and Safety Scope

Is `<memory>`, the strongest lever for making the agent *reject* a wrong claim, symmetrically the strongest lever for making it *accept* one? On tasks the agent already solves, we inject a fresh c_* through each external role and count an attack as successful when the final answer adopts c_* . Across five experiments crossing two models, two domains, and calculator-available versus no-tools regimes, all 20 cells stay at or below the 3.3% baseline error rate (Table 9). The forward lift has no symmetric reverse: the agent commits to a wrong claim in its own `<thought>` about 83% of the time but to a byte-identical claim in an external role at most 3.3%, the same self-versus-external asymmetry the intervention exploits, now read in the opposite direction. This is the mirror image of indirect prompt injection (Greshake et al. 2023; Bagdasaryan et al. 2023): the load-bearing untrusted content is the agent’s own intermediate, not external input.

This safety is a default, not a guarantee. Holding the `<memory>` wrapper fixed and varying only the trailing audit, a single sentence ordering the agent to treat the memory as truth and not verify raises the attack rate from 3.3% to 70%, while a distrust framing leaves it unchanged (Table 10). Verify-by-default is therefore a learned response to the prompt context, not an architectural guarantee, which is why the intervention is a reliability lever rather than a hardened defense: a deployment that does not control the user-prompt against trust framing loses the safety property.

5 Discussion

5.1 What “Self” Means in a Chat-Template

We interpret the effect as a by-product of instruction tuning, which rewards responding to content arriving under external roles (user, system, tool) far more than acting on the model’s own `<thought>` block as a target. Under

Table 9: Adversarial mirror across two models, two domains, and two verification regimes ($n = 30$ per cell). Values are attack-success rates; A0 is the no-injection baseline.

Experiment	A0	USER	TOOL	MEM.
Qwen-72B math + calc	3.3%	3.3%	0.0%	3.3%
Qwen-72B math no-tool	3.3%	3.3%	3.3%	3.3%
Llama-70B math + calc	3.3%	3.3%	3.3%	3.3%
Qwen-72B logic	0.0%	0.0%	0.0%	0.0%
Llama-70B logic	0.0%	3.3%	0.0%	0.0%

Table 10: Instructability of the `<memory>` safety on Qwen-72B math ($n = 30$). Audit framing is varied with the wrapper fixed; values are attack-success rates.

Condition	Attack	Δ
M_DEFAULT (<code><memory></code> , neutral audit)	3.3%	—
M_TRUST_COMMAND (“treat as truth, do not verify”)	70.0%	+66.7pp
M_DISTRICT (“verify before using”)	3.3%	± 0 pp

this interpretation, the instruction to “audit your reasoning” admits no well-practiced response other than re-derivation, which is what the control of Sec. 4.3 observes: the agent recomputes but rarely singles out and rejects the specific intermediate. The relabel sidesteps the gap by recasting an unaddressable thought-internal claim as the kind of object the model handles fluently, an external assertion that invites a verify-and-rebut response. This interpretation also predicts the model- and domain-dependence of the strongest wrapper: the strongest handle is the role whose discourse register best matches the wrong claim’s content type, since that match recruits the most fluent rebuttal. The “self” that fails to self-correct is, on this view, not a cognitive deficit but a chat-template artifact.

5.2 Reconciliation with Prior Work

The result connects four lines that have been studied separately. The unfaithful-CoT literature (Turpin et al. 2023; Lanham et al. 2023; Chen et al. 2025b) shows that a reasoning trace can diverge from the computation behind the answer; we add its corrective face, showing the trace is also not addressable as a discrete object even when the agent can still revise its content. The user-assistant bias of Pan et al. (2026) captures the same role-asymmetry in static completion; we carry it into the corrective regime and turn it into a reliability lever rather than only a vulnerability. The memory-poisoning literature (Dong et al. 2026; Park et al. 2023) reports high attack success from narrative-coherent payloads; our null adversarial result occupies the complementary regime of bare, incoherent injection, and the instructability experiment shows that trust framing, not the role tag, is the gateway. Finally, Wallace et al. (2024) predict that a `system` wrapper outranks `user` content; we show this priority is itself overridable from inside the user role, recasting role priority as behavioral rather than architectural.

5.3 Scope and Limitations

The lift is largest on vanilla instruction-tuned models with non-ceiling baselines; reasoning-tuned families (gpt-oss-20B; DeepSeek-R1 reaching 100% under audit alone, Appendix D) and near-ceiling frontier vanilla models (Gemini 2.5 Flash; GPT-4.1 in Appendix B.2) leave little headroom, which the addressability account itself predicts: a model that already invests deliberate verification under audit-only does not need an external referent. Cell-level samples are $n = 30$ paired binary outcomes, sufficient for the headline effect sizes but tight for fine-grained subgroup analysis. The verifiable-task focus (GSM8K-style math, generated logical deduction, BBH-LD) is what allows the strict-identification judge to operate; extension to free-form long-form reasoning, code debugging, and multi-step planning is the natural next step. The asymmetric safety degrades to a 70% attack rate under a single trust-framing prompt (Sec. 4.5), so the intervention is a reliability lever, not a hardened defense, and any deployment that does not control the surrounding user-prompt context against attacker-controlled trust framing loses the safety property. The mechanism is established behaviorally: a hidden-state probe trained on the model’s final-layer embedding is null (Appendix C.3), and a circuit-level confirmation via attention patterns or activation patching remains an open direction. Three further scope notes apply. First, the failure pool is built by selecting tasks on which the audit-only baseline already fails the strict-identification criterion, so the headline lifts speak to the targeted regime and not to in-the-wild prevalence. Second, the `L_TOOL` condition surfaces a calculator-tool return whose tokenization varies materially across chat templates, so a small share of the cross-family variance in `L_TOOL` reflects template-level rather than role-level differences; chat-template specs for each model appear in Appendix E. Third, closed-weight cells (GPT-4o, Claude Sonnet 4, GPT-4.1) ran with n below 30 due to free-tier rate limits, so their CIs are correspondingly wider and the cross-method ordering on those cells should be read as suggestive rather than confirmatory. None of these bounds undercuts the behavioral effect itself.

6 Conclusion

An LLM agent’s failure to self-correct a wrong claim inside its own `<thought>` is, to a large degree, not a reasoning deficit but a chat-template artifact: re-presenting the byte-identical claim under an external role lifts the correction rate by 23 to 93 pp across seven model families and three domains, while a self-distrust prompt that leaves the claim in place does not. Our experiments isolate the mechanism as addressability rather than verification: the agent retains the capability to check the claim but has no learned way to act on a thought-internal substring as a discrete object, and the relabel supplies the missing handle. This reframing turns a studied limitation into a controllable, zero-training reliability lever, bounded by an asymmetric safety that a single trust-framing sentence can override. Whether addressability also governs self-correction in free-form reasoning, beyond the verifiable tasks studied here, is the natural next question, and one we expect the same role-handle to illuminate.

References

- Abdin, M.; Aneja, J.; Behl, H.; Bubeck, S.; Eldan, R.; Gunasekar, S.; Harrison, M.; Hewett, R.; Javaheripi, M.; Kauffmann, P.; Lee, J.; Lee, Y. T.; Li, Y.; Liu, W.; Mendes, C.; Nguyen, A.; Price, E.; de Rosa, G.; Saarikivi, O.; and Zhang, Y. 2024. *Phi-4 Technical Report*.
- Bagdasaryan, E.; Hsieh, T.-Y.; Nassi, B.; and Shmatikov, V. 2023. Abusing Images and Sounds for Indirect Instruction Injection in Multi-Modal LLMs. arXiv:2307.10490.
- Bhatia, G.; Sripada, S. G.; Allan, K.; and Azcona, J. 2026. Distributional Semantics Tracing: A Framework for Explaining Hallucinations in Large Language Models. arXiv:2510.06107.
- BIG-bench Collaboration. 2023. Beyond the Imitation Game: Quantifying and Extrapolating the Capabilities of Language Models. arXiv:2206.04615.
- Chen, J.; He, Q.; Yuan, S.; Chen, A.; Cai, Z.; Dai, W.; Yu, H.; Chen, J.; Li, X.; Yu, Q.; Zhou, H.; and Wang, M. 2025a. Enigmata: Scaling Logical Reasoning in Large Language Models with Synthetic Verifiable Puzzles. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Chen, X.; Lin, M.; Schärli, N.; and Zhou, D. 2023. Teaching Large Language Models to Self-Debug. arXiv:2304.05128.
- Chen, Y.; Benton, J.; Radhakrishnan, A.; Uesato, J.; Denison, C.; Schulman, J.; Somani, A.; Hase, P.; Wagner, M.; Roger, F.; Mikulik, V.; Bowman, S. R.; Leike, J.; Kaplan, J.; and Perez, E. 2025b. Reasoning Models Don't Always Say What They Think. arXiv:2505.05410.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. *ArXiv*.
- Comanici, G.; Bieber, E.; Schaeckermann, M.; Pasupat, I.; Sachdeva, N.; Dhillon, I.; Blistein, M.; Ram, O.; Zhang, D.; Rosen, E.; Marris, L.; Petulla, S.; Gaffney, C.; Aharoni, A.; Lintz, N.; Pais, T.; Jacobsson, H.; Szpektor, I.; Jiang, N.-J.; and Hahn, C. 2025. *Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities*.
- Dhuliawala, S.; Komeili, M.; Xu, J.; Raileanu, R.; Li, X.; Celikyilmaz, A.; and Weston, J. 2023. Chain-of-Verification Reduces Hallucination in Large Language Models. arXiv:2309.11495.
- Dong, S.; Xu, S.; He, P.; Li, Y.; Tang, J.; Liu, T.; Liu, H.; and Xiang, Z. 2026. Memory Injection Attacks on LLM Agents via Query-Only Interaction. arXiv:2503.03704.
- Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; Goyal, A.; Hartshorn, A.; Yang, A.; Mitra, A.; Sravankumar, A.; Korenev, A.; Hinsvark, A.; Rao, A.; Zhang, A.; and Zhao, Z. 2024. *The Llama 3 Herd of Models*.
- Gao, L.; Madaan, A.; Zhou, S.; Alon, U.; Liu, P.; Yang, Y.; Callan, J.; and Neubig, G. 2023. PAL: Program-aided Language Models. arXiv:2211.10435.
- Gemma Team. 2025. Gemma 3 Technical Report. arXiv:2503.19786.
- Gou, Z.; Shao, Z.; Gong, Y.; Shen, Y.; Yang, Y.; Duan, N.; and Chen, W. 2023. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *The Twelfth International Conference on Learning Representations*.
- Greshake, K.; Abdelnabi, S.; Mishra, S.; Endres, C.; Holz, T.; and Fritz, M. 2023. Not What You've Signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173.
- Huang, J.; Chen, X.; Mishra, S.; Zheng, H. S.; Yu, A. W.; Song, X.; and Zhou, D. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. arXiv:2310.01798.
- Ji, Z.; Lee, N.; Frieske, R.; Yu, T.; Su, D.; Xu, Y.; Ishii, E.; Bang, Y.; Chen, D.; Dai, W.; Chan, H. S.; Madotto, A.; and Fung, P. 2022. Survey of Hallucination in Natural Language Generation. <https://arxiv.org/abs/2202.03629v7>.
- Kamoi, R.; Zhang, Y.; Zhang, N.; Han, J.; and Zhang, R. 2024. When Can LLMs Actually Correct Their Own Mistakes? A Critical Survey of Self-Correction of LLMs. arXiv:2406.01297.
- Kim, S.; and Khashabi, D. 2025. Challenging the Evaluator: LLM Sycophancy Under User Rebuttal. arXiv:2509.16533.
- Kojima, T.; Gu, S. S.; Reid, M.; Matsuo, Y.; and Iwasawa, Y. 2023. Large Language Models Are Zero-Shot Reasoners. arXiv:2205.11916.
- Lanham, T.; Chen, A.; Radhakrishnan, A.; Steiner, B.; Denison, C.; Hernandez, D.; Li, D.; Durmus, E.; Hubinger, E.; Kernion, J.; Lukošiušė, K.; Nguyen, K.; Cheng, N.; Joseph, N.; Schiefer, N.; Rausch, O.; Larson, R.; McCandlish, S.; Kundu, S.; Kadavath, S.; Yang, S.; Henighan, T.; Maxwell, T.; Telleen-Lawton, T.; Hume, T.; Hatfield-Dodds, Z.; Kaplan, J.; Brauner, J.; Bowman, S. R.; and Perez, E. 2023. Measuring Faithfulness in Chain-of-Thought Reasoning. arXiv:2307.13702.
- Lin, B. Y.; Bras, R. L.; Richardson, K.; Sabharwal, A.; Poovendran, R.; Clark, P.; and Choi, Y. 2025. ZebraLogic: On the Scaling Limits of LLMs for Logical Reasoning. In *Forty-Second International Conference on Machine Learning*.
- Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651.
- Olausson, T. X.; Inala, J. P.; Wang, C.; Gao, J.; and Solar-Lezama, A. 2024. Is Self-Repair a Silver Bullet for Code Generation? arXiv:2306.09896.
- OpenAI. 2024. GPT-4o System Card. <https://arxiv.org/abs/2410.21276v1>.
- Pan, X.; Fan, J.; Xiong, Z.; Hahami, E.; Overwiening, J.; and Xie, Z. 2026. User-Assistant Bias in LLMs. arXiv:2508.15815.
- Park, J. S.; O'Brien, J. C.; Cai, C. J.; Morris, M. R.; Liang, P.; and Bernstein, M. S. 2023. Generative Agents: Interactive Simulacra of Human Behavior. arXiv:2304.03442.

- Perez, E.; Ringer, S.; Lukošiušė, K.; Nguyen, K.; Chen, E.; Heiner, S.; Pettit, C.; Olsson, C.; Kundu, S.; Kadavath, S.; Jones, A.; Chen, A.; Mann, B.; Israel, B.; Seethor, B.; McKinnon, C.; Olah, C.; Yan, D.; Amodei, D.; Amodei, D.; Drain, D.; Li, D.; Tran-Johnson, E.; Khundadze, G.; Kernion, J.; Landis, J.; Kerr, J.; Mueller, J.; Hyun, J.; Landau, J.; Ndousse, K.; Goldberg, L.; Lovitt, L.; Lucas, M.; Sellitto, M.; Zhang, M.; Kingsland, N.; Elhage, N.; Joseph, N.; Mercado, N.; DasSarma, N.; Rausch, O.; Larson, R.; McCandlish, S.; Johnston, S.; Kravec, S.; Showk, S. E.; Lanham, T.; Telleen-Lawton, T.; Brown, T.; Henighan, T.; Hume, T.; Bai, Y.; Hatfield-Dodds, Z.; Clark, J.; Bowman, S. R.; Askell, A.; Grosse, R.; Hernandez, D.; Ganguli, D.; Hubinger, E.; Schiefer, N.; and Kaplan, J. 2022. Discovering Language Model Behaviors with Model-Written Evaluations. arXiv:2212.09251.
- Qwen. 2025. Qwen2.5 Technical Report. arXiv:2412.15115.
- Schick, T.; Dwivedi-Yu, J.; Dessi, R.; Raileanu, R.; Lomeli, M.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761.
- Sharma, M.; Tong, M.; Korbak, T.; Duvenaud, D.; Askell, A.; Bowman, S. R.; Cheng, N.; Durmus, E.; Hatfield-Dodds, Z.; Johnston, S. R.; Kravec, S.; Maxwell, T.; McCandlish, S.; Ndousse, K.; Rausch, O.; Schiefer, N.; Yan, D.; Zhang, M.; and Perez, E. 2025. Towards Understanding Sycophancy in Language Models. arXiv:2310.13548.
- Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K. R.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Thirty-Seventh Conference on Neural Information Processing Systems*.
- Stechly, K.; Marquez, M.; and Kambhampati, S. 2023. GPT-4 Doesn't Know It's Wrong: An Analysis of Iterative Prompting for Reasoning Problems. arXiv:2310.12397.
- Suzgun, M.; Scales, N.; Schärli, N.; Gehrmann, S.; Tay, Y.; Chung, H. W.; Chowdhery, A.; Le, Q. V.; Chi, E. H.; Zhou, D.; and Wei, J. 2022. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. arXiv:2210.09261.
- Turpin, M.; Michael, J.; Perez, E.; and Bowman, S. R. 2023. Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting. arXiv:2305.04388.
- Tyen, G.; Mansoor, H.; Cărbune, V.; Chen, P.; and Mak, T. 2024. LLMs Cannot Find Reasoning Errors, but Can Correct Them given the Error Location. arXiv:2311.08516.
- Wallace, E.; Xiao, K.; Leike, R.; Weng, L.; Heidecke, J.; and Beutel, A. 2024. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208.
- Wang, X.; Wei, J.; Schuurmans, D.; Le, Q.; Chi, E.; Narang, S.; Chowdhery, A.; and Zhou, D. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171.
- Wei, J.; Huang, D.; Lu, Y.; Zhou, D.; and Le, Q. V. 2024. Simple Synthetic Data Reduces Sycophancy in Large Language Models. arXiv:2308.03958.
- Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E.; Le, Q.; and Zhou, D. 2023. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903.
- Wei, Q.; Yang, T.; Wang, Y.; Li, X.; Li, L.; Yin, Z.; Zhan, Y.; Holz, T.; Lin, Z.; and Wang, X. 2025. A-MemGuard: A Proactive Defense Framework for LLM-Based Agent Memory. arXiv:2510.02373.
- Welleck, S.; Lu, X.; West, P.; Brahman, F.; Shen, T.; Khashabi, D.; and Choi, Y. 2022. Generating Sequences by Learning to Self-Correct. In *The Eleventh International Conference on Learning Representations*.
- Yao, S.; Yu, D.; Zhao, J.; Shafran, I.; Griffiths, T. L.; Cao, Y.; and Narasimhan, K. 2023a. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. arXiv:2305.10601.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023b. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629.
- Yin, C.; Sha, Z.; Cui, S.; Meng, C.; and Li, Z. 2026. The Reasoning Trap: How Enhancing LLM Reasoning Amplifies Tool Hallucination. arXiv:2510.22977.
- Zhang, Y.; Li, Y.; Cui, L.; Cai, D.; Liu, L.; Fu, T.; Huang, X.; Zhao, E.; Zhang, Y.; Chen, Y.; Wang, L.; Luu, A. T.; Bi, W.; Shi, F.; and Shi, S. 2025. Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models. *Computational Linguistics*, 51(4): 1373–1418.
- Zhang, Z.; Bo, X.; Ma, C.; Li, R.; Chen, X.; Dai, Q.; Zhu, J.; Dong, Z.; and Wen, J.-R. 2024. A Survey on the Memory Mechanism of Large Language Model Based Agents. arXiv:2404.13501.
- Zhao, X.; Xu, T.; Wang, X.; Chen, Z.; Jin, D.; Tan, L.; Yen-Ting; Yu, Z.; Zhao, Z.; He, Y.; Wang, S.; Fang, H.; Chandar, S.; and Zhu, C. 2025. Boosting LLM Reasoning via Spontaneous Self-Correction. arXiv:2506.06923.
- Zou, A.; Wang, Z.; Carlini, N.; Nasr, M.; Kolter, J. Z.; and Fredrikson, M. 2023. Universal and Transferable Adversarial Attacks on Aligned Language Models. arXiv:2307.15043.

A Statistical Robustness

A.1 Multiple-Comparison Correction

The main 13-cell table reports 52 paired contrasts (13 cells $\times$ 4 relabel conditions). We apply Holm-Bonferroni at family-wise error rate $\alpha = 0.05$ and Benjamini-Hochberg at false-discovery rates 0.05 and 0.01. Of the 52 contrasts, 28 survive Holm-Bonferroni FWER = 0.05, 32 survive BH FDR = 0.05, and 30 survive BH FDR = 0.01. At the cell level, 9 of 13 cells retain at least one significant contrast under Holm-Bonferroni. The cells that lose all significance are the three ceiling cells (gpt-oss-20B math, Gemini 2.5 Flash math, Qwen-72B BBH-LD) plus Gemini 2.5 Flash-Lite, whose only positive contrast was the borderline L_MEMORY entry at $\Delta = +23.3$ pp. The headline finding (10 of 13 cells significant before correction; 9 of 13 after Holm-Bonferroni) is therefore robust to strict multiple-testing correction.

A.2 Failure-Pool Stochasticity

The L0_SELF baseline is measured at $T = 0$ with a single seed, so we ask whether the +53 pp lift on L_MEMORY could be partially explained by stochastic re-rolling rather than the role manipulation. We test directly. On the same 30 Qwen-72B math failure-pool tasks, we re-run L0_SELF at $T = 0.5$ with five independent seeds, holding the agent’s prior trajectory and c_* fixed. The aggregate re-roll correction rate lands at 16.0% (95% CI [8.7, 24.7]), indistinguishable from the main-run L0_SELF of 16.7% (per-seed CR 10.0–20.0%). The L_MEMORY gain over the re-roll baseline is +54.0 pp, slightly *larger* than over the original single-seed baseline. The lift in Sec. 4.2 is not attributable to stochastic re-rolling.

A.3 Multi-Judge Robustness

The headline analyses use a locked Qwen-72B judge. This judge could in principle be biased toward Qwen-style explicit identification, so that responses from non-Qwen agents (Llama, GPT-4o, Claude, Gemini) are judged differently. To test this, we stratify a sample of 200 trials (40 per source) across five model classes (Qwen-72B math, Llama-3.3-70B math, GPT-4o math, Claude Sonnet 4 math, Gemini 2.5 Flash-Lite math) and re-judge each trajectory with a second judge (Llama-3.3-70B) using the same locked judge prompt.

Two judges agree on 92.5% of 200 trials with Cohen’s $\kappa = 0.843$. The confusion matrix is dominated by joint-YES (115) and joint-NO (70); disagreements are asymmetric, with Qwen-72B labeling YES while Llama-3.3-70B labels NO 14 times, and the reverse only 1 time. Qwen-72B is therefore the slightly more generous judge, so swapping in Llama-3.3-70B as the canonical judge would lower a few correction rates but preserve rank ordering. The strict-identification criterion is judge-invariant.

A.4 Final-Answer Correctness as a Secondary Metric

The strict-identification criterion used throughout the paper measures whether the agent explicitly names c_* and rejects it. By construction this metric may under-credit silent re-derivation or, conversely, mask a final-answer regression. To

Table 11: Multi-judge robustness on 200 stratified trials. Two judges (Qwen-72B and Llama-3.3-70B) re-judge the same trajectories with the same locked prompt; agreement is computed on the strict-identification verdict (YES/NO).

Source	n	Agreement
Qwen-72B main	40	82.5%
Llama-3.3-70B main	40	97.5%
GPT-4o main	40	95.0%
Claude Sonnet 4 main	40	97.5%
Gemini 2.5 Flash-Lite	40	90.0%
Overall	200	92.5%
Cohen’s $\kappa = 0.843$ (near-perfect agreement)		

Table 12: Strict-identification CR vs. judge-verified final-answer correctness (%) on the two 70B-class math failure pools ($n = 30$ paired). Δ final is the paired contrast of final CR vs. L0_SELF; all $p > 0.05$ (paired bootstrap, two-sided).

Cell	Cond.	strict	final	Δ final
Qwen-72B	L0_SELF	16.7	76.7	—
	L_USER_WAIT	40.0	80.0	+3.3
	L_USER_NEUTRAL	43.3	86.7	+10.0
	L_TOOL	43.3	76.7	± 0
	L_MEMORY	70.0	80.0	+3.3
Llama-70B	L0_SELF	0.0	70.0	—
	L_USER_WAIT	56.7	73.3	+3.3
	L_USER_NEUTRAL	60.0	80.0	+10.0
	L_TOOL	3.3	86.7	+16.7
	L_MEMORY	86.7	56.7	−13.3

probe both, we re-score every Qwen-72B math and Llama-3.3-70B math trial with the locked Qwen-72B judge under a different prompt: “given the gold answer X , did the response state X as its final answer, regardless of phrasing?”. We use a judge rather than a regex because reworded phrasings such as “the correct answer is 42” or “so it’s 42” miss a literal “Final Answer: 42” match yet still arrive at the correct numeric value, and the miss is not uniform across conditions.

Three observations follow. First, on both cells the audit-only baseline already reaches the correct final answer in 70 to 77% of trials despite a strict CR of 0 to 17%, evidencing strong silent re-derivation that the strict criterion explicitly does not credit. Second, across all eight relabel-vs-baseline contrasts the paired final-CR difference falls in $[-13, +17]$ pp and none is significant at $p < 0.05$; the strict-CR lifts do not come at the cost of final-answer accuracy. Third, the two metrics rank conditions differently: on Llama, L_TOOL has the highest final CR (86.7%) but a near-zero strict CR (3.3%), and L_MEMORY reverses this with the highest strict CR (86.7%) and the lowest final CR (56.7%, paired $\Delta = -13.3$ pp, CI $[-33, +7]$). The non-significant negative trend on L_MEMORY is consistent with a known failure mode: an agent that explicitly commits to a replacement value for c_* occasionally locks in an incorrect candidate, whereas silent re-derivation is free to re-roll. The two metrics are therefore complementary; the main paper reports

Table 13: Comparison with published self-correction methods on the Qwen-72B and Llama-3.3-70B math failure pools ($n = 30$ paired per condition). Entries are correction-rate % with significance vs. the audit-only baseline.

Model	audit	S.-Refine	Reflex.	CoVe	L-MEM.
Qwen-72B	30.0	6.7	46.7*	10.0	80.0***
Llama-70B	3.3	0.0	6.7	0.0	86.7***

strict CR because the addressability question is the one the paper is making, and the absence of a significant final-CR regression bounds the deployment risk.

B Comparison with Published Self-Correction Methods

We test whether the lift observed in the main table replicates when published intrinsic self-correction protocols are run on the same failure pool under the same locked judge. Three methods are compared against the audit-only baseline and the role-relabel reference: Self-Refine (Madaan et al. 2023), Reflexion (Shinn et al. 2023), and Chain-of-Verification (CoVe) (Dhuliawala et al. 2023).

Protocols. The Self-Refine critique instruction is taken verbatim from the Madaan et al. GSM8K release, adapted by substituting “reasoning” for “code” since our solutions are natural-language traces rather than PaL-style code. Reflexion is run as a two-cycle reflect-and-retry following the HotpotQA pattern. CoVe is implemented as the three-step plan/execute/audit pipeline: the agent first plans verification questions about its intermediate values, each question is answered independently with a fresh-context call, and the verification answers are surfaced before the trailing audit. All four methods (the three baselines plus L-MEMORY) follow the same audit prompt; only the wrapping context differs.

B.1 Qwen-72B and Llama-3.3-70B Math

Table 13 reports the four-method comparison on the Qwen-72B and Llama-3.3-70B math failure pools. Self-Refine and CoVe produce *negative* lifts under the strict-identification criterion: both methods induce generic re-derivation that arrives at correct final answers but never names the wrong intermediate, so the locked judge returns NO. Reflexion produces a small positive lift on Qwen but collapses on Llama. The role-relabel reference dominates on both cells.

To preserve strict paired comparison across the four methods, the audit-only condition and L-MEMORY were re-run within this batch rather than imported from the main table. Batch-level non-determinism in the Ollama runtime at $T = 0$ causes their absolute values to drift slightly from the canonical main-table figures: on Qwen-72B the canonical $L_0 = 16.7\%$ and L-MEMORY = 70.0% become 30.0% and 80.0% in this batch. The within-batch L-MEMORY-vs-audit paired contrast is +50.0 pp, in agreement with the canonical paired lift of +53.3 pp within bootstrap CI. The ordering of the four methods and the dominance of L-MEMORY are preserved across both batches.

Table 14: Frontier replication of the four-method comparison on GPT-4o ($n \approx 18$) and GPT-4.1 ($n \approx 17$, ceil.) math. Entries are correction-rate % with significance vs. the audit-only baseline.

Model	audit	S.-Refine	Reflex.	CoVe	L-MEM.
GPT-4o	31.6	63.2**	50.0*	11.1	72.2***
GPT-4.1	82.4	76.5	100.0*	35.3	93.8

Table 15: Correction rate under different system-role tag wrappers on Qwen-72B math ($n = 30$ paired). The nonsense control <xqzy> retains 30.0% CR (+13 over L0_SELF), and <memory> dominates the other tags.

Tag	CR	vs. <memory>
<memory>	70.0%	—
<retrieved>	63.3%	−6.7 (ns)
<context>	53.3%	−16.7
<note>	50.0%	−20.0
<reference>	53.3%	−16.7
<xqzy> (nonsense)	30.0%	−40.0 ($p = 0.0002$)
(reference) L0_SELF	16.7%	—

B.2 Frontier Replication on GPT-4o and GPT-4.1

We replicate the comparison on two frontier vanilla-instruct models, served via the GitHub Models API. Per-condition n is smaller than 30 because free-tier rate limits cap how many trials can be run within the experimental window.

On GPT-4o the four-method ordering replicates the role-relabel as the strongest single intervention (+52.9 pp, $p < 0.001$), with Self-Refine and Reflexion now also producing significant positive lifts (+31.6 and +22.2 pp respectively) and CoVe still hurting. On GPT-4.1 the audit-only baseline already sits at 82.4% — the same ceiling regime as Gemini 2.5 Flash in the main table—so all methods cluster near the ceiling except CoVe, which catastrophically drops to 35.3% (−47.1 pp) by fragmenting the response into verification questions that the strict judge does not credit.

Take-away. Under the strict-identification criterion, the role-relabel intervention dominates the published baselines on every non-ceiling cell tested. The pattern is consistent with the addressability account: methods that drive the agent through additional reasoning passes solve the underlying arithmetic but fail to address the wrong intermediate as a discrete entity.

C Mechanism Triangulation

C.1 Role-Token Variants

The handle-granularity ladder of Sec. 4.2 isolates the role label from a bare syntactic wrapper but uses the canonical <memory> tag throughout. To separate the contribution of the system-role token pair from the memory-aligned semantic prior on the tag, we vary the tag while holding the role (system) and the wrapped content (c_*) constant.

Table 16: First-token mean probability mass on *agreement* tokens (Indeed, Yes, So, ...) by role context, across four open-weight families on the same 15-task failure-pool subset.

Model	L_THOUGHT	L_USER_NEUT.	L_TOOL	L_MEMORY
Qwen-72B (reference)	0.001	0.0001	0.0005	0.226
Llama-3.3-70B	0.019	0.0001	0.017	0.004
Phi-4-14B	0.212	0.0004	0.019	0.019
Gemma-3-12B	0.005	0.067	0.803	~ 0

The pattern decomposes into two layers. The bare system-role tag pair is itself worth a non-trivial lift: even the gibberish `<xqzy>` reaches 30.0%, which is +13.3 pp above the main L0_SELF baseline. The lexical identity of the tag then carries a further share: `<memory>` dominates the gibberish control by 40 pp ($p = 0.0002$, paired bootstrap), and `<memory>` and the closest semantic neighbour `<retrieved>` are statistically indistinguishable. Combined with the H0-H4 ladder in the main text, the picture is: a syntactic boundary alone supplies a baseline lift, a system-role wrapper amplifies it, and a memory-aligned semantic prior on the tag pushes it to the full +53 pp.

C.2 First-Token Signature Across Model Families

The engage-then-verify first-token signature reported in Sec. 4.3 is on Qwen-72B. To check whether the specific lexical mode generalizes, we replicate the logprob experiment on three additional vanilla-instruct families: Llama-3.3-70B, Phi-4-14B, and Gemma-3-12B. Reasoning-mode models are excluded by construction, since a trained reasoning preamble (e.g. Qwen3-30B’s “Okay”) deterministically occupies the first-token slot and leaves no stance signal to read. Table 16 reports the agreement-token mass under each role context.

The lexical first-token mode is markedly model-specific. The L_MEMORY agreement spike that defines the Qwen-72B signature does not reproduce in the other three families: Gemma-3 concentrates agreement mass under L_TOOL, Phi-4 under L_THOUGHT, and Llama-3.3-70B produces challenge tokens uniformly.

The honest reading is that the engage-then-verify signature in its specific lexical form is a Qwen-72B feature. The *behavioral* lift (Table 2) is universal, but the first-token route through which different families reach the rejection differs by tokenizer and training mix.

C.3 Hidden-State Probing

We test whether the relabel produces a representation change that is linearly readable from the model’s final-layer embedding. For each of the 30 Qwen-72B math failure-pool tasks and each of the five conditions, we render the trailing message slice (the role-wrapped c_* plus the audit prompt) and embed it via the served model’s embedding endpoint, yielding a 30×5 matrix of 8192-dimensional vectors paired with the binary correction outcome.

A leave-one-task-out logistic probe trained to predict correction from the embedding achieves 0.573 accuracy, exactly matching the majority-class baseline. The linear probe

is null: the final-layer mean-pooled embedding does not linearly carry the per-trial outcome. A weaker positive signal remains. The per-task cosine distance from L0_SELF is ~ 0.02 for all four relabel conditions, indicating that the relabel does shift the representation. The sharpest pairwise centroid signal is L_MEMORY vs. L_TOOL (0.005); other external-role pair centroids are within the range of L0_SELF-to-external centroid distances (0.009 to 0.012).

The representation does shift under the relabel, but a deeper probing protocol – per-layer or activation-patching with the model loaded in a deep-learning framework with adequate GPU memory – would be needed to localize that change. We did not have such access in our setup, and report this as a partial mechanistic finding.

C.4 Within-Thought Duplication and Recency Control

The canonical L_MEMORY condition appends a byte-identical copy of c_* as a new system `<memory>` message, so the second copy of c_* both raises its overall count in the prompt (duplication) and occupies the most recent message position before the audit (recency). Either confound, rather than the role tag itself, could in principle drive the +53.3 pp main lift on Qwen-72B math. We isolate the role-tag claim with a three-way comparison that holds duplication count and recency position constant while varying only the role of the second copy.

D0_BASELINE: audit-only; c_* appears once, inside the original `<thought>` block (equivalent to L0_SELF).

D_INTHOUGHT_DUP: a byte-identical second copy of c_* is appended to the assistant turn as `<thought> $c_*$ </thought>`, so c_* now appears twice, both inside `<thought>`, with the second copy occupying the same temporal position as L_MEMORY’s appended message.

D_MEMORY_REF: the canonical L_MEMORY condition, re-run inside this batch for paired comparison; c_* appears twice, once in `<thought>` and once in system `<memory>`.

Across these three conditions the number of c_* appearances (1, 2, 2), the position of the second appearance (none, end-of-trajectory, end-of-trajectory), and the byte content of every copy are matched between D_INTHOUGHT_DUP and D_MEMORY_REF; the only variable is the role of the second copy.

Three observations follow. First, duplicating c_* inside `<thought>` alone produces only a +6.7 pp shift over the audit-only baseline (CI [0.0, +16.7], $p = 0.26$), which is neither distinguishable from zero nor remotely comparable to the canonical +53.3 pp lift. Second, the canonical L_MEMORY value of 70.0% reproduces the main-table number within this batch, providing an internal consistency check. Third, and most importantly, the paired contrast between D_MEMORY_REF and D_INTHOUGHT_DUP is +46.7 pp ($p < 0.001$, CI [+26.7, +66.7]); since these two conditions are matched on duplication count, recency position, and byte content, this contrast isolates the pure role-tag effect. A duplication- or salience-only account of the main

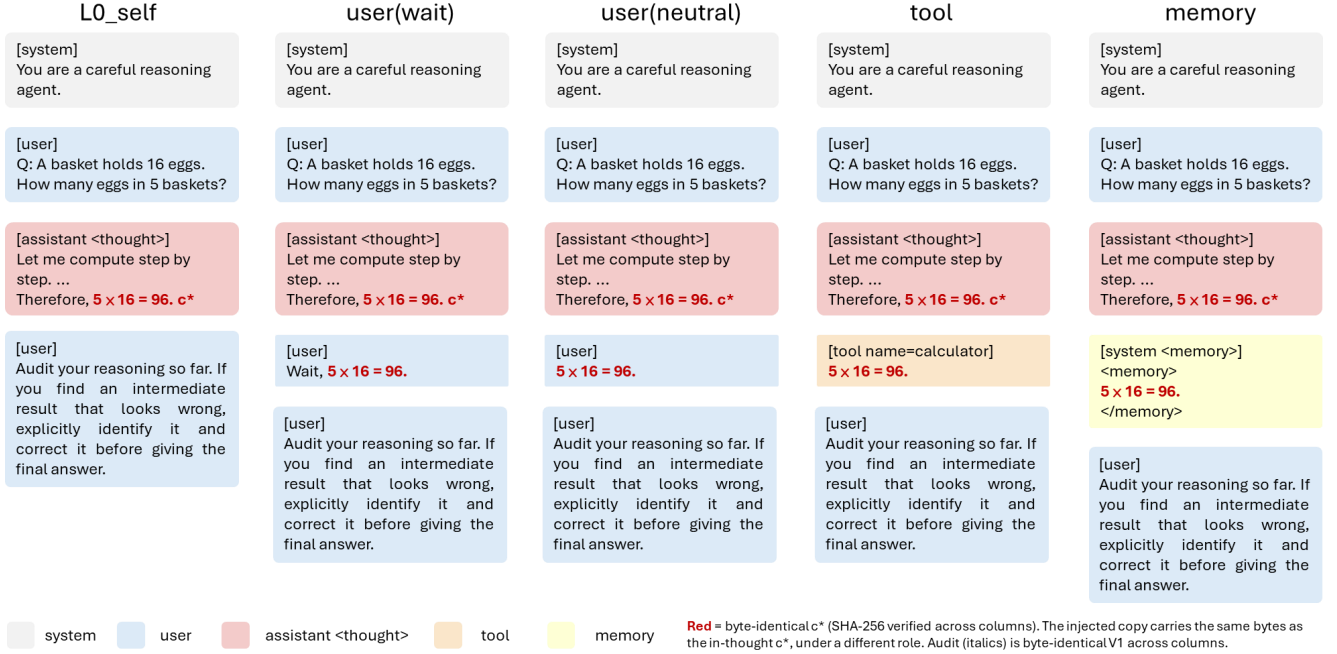


Figure 3: The five chat-template conditions rendered verbatim. Red text is the byte-identical c_* (SHA-256 verified); the audit (italics, bottom) is byte-identical V1 across columns.

Table 17: Within-thought duplication and recency control on the Qwen-72B math failure pool ($n = 30$ paired). Δ values are paired contrasts vs. D0_BASELINE; p is from a two-sided 10,000-sample paired bootstrap.

Condition	CR	Δ vs. D0
D0_BASELINE (audit only)	16.7%	—
D_INTHOUGHT_DUP (c_* twice in <thought>)	23.3%	+6.7 ($p=0.26$)
D_MEMORY_REF (canonical L_MEMORY)	70.0%	+53.3***
Key contrast: D_MEMORY_REF — D_INTHOUGHT_DUP = +46.7 pp ($p<0.001$).		

lift is incompatible with these numbers, and any residual recency contribution can supply at most +6.7 pp—roughly an eighth of the canonical effect.

D Reasoning-Model Scope

The main table identifies three ceiling cells in which the role-relabel does not add (gpt-oss-20B at $L_0 = 77\%$, Gemini 2.5 Flash at $L_0 = 97\%$, Qwen-72B BBH-LD at $L_0 = 67\%$). Two of these (gpt-oss-20B and Qwen3-30B in reasoning mode) are reasoning-tuned models. We add a frontier-scale reasoning-model data point with DeepSeek-R1, served via GitHub Models. DeepSeek-R1 emits explicit <think>...</think> traces before its final answer.

Free-tier rate limits cap per-cell n in our window. We collected 57 trials in total, distributed across the five

conditions: L0_SELF ($n = 12$), L_USER_WAIT ($n = 13$), L_USER_NEUTRAL ($n = 11$), L_TOOL ($n = 10$), L_MEMORY ($n = 11$). Every observed trial under every condition is correct; the per-condition CR is 100% across the board. DeepSeek-R1 reaches the ceiling under audit-only alone, leaving no headroom for the relabel. The same pattern is seen on GPT-4.1 in Appendix B.2 ($L_0 = 82.4\%$).

The honest scope of the role-relabel finding is therefore *vanilla instruct models with non-ceiling baselines*: when a model already invests deliberate verification under audit-only (whether through built-in reasoning traces or through strong instruction tuning), the relabel does not add. This is consistent with the addressability account, since a model that already addresses its own intermediate as a verifiable object does not need an external referent to grip.

E Chat-Template and Prompt Templates

Role-token implementation differs substantially across model families: the same logical role (system, user, tool) is realized by entirely different special-token sequences depending on each model’s training-time chat template. This heterogeneity is precisely why the cross-family coverage of Table 2 matters, since a lift that survives such varied tokenizations cannot be an artifact of any single template, and it is also why the role-handle abstraction must be documented at the token level rather than assumed uniform. We therefore record the exact begin/end role-tag pairs used for every model, so that the byte-identity guarantee of Sec. 3.2 is auditable down to the template boundary characters.

Figure 3 renders all five conditions verbatim on a repre-

sentative GSM8K-style task with c_* “ $5 \times 16 = 96$ ”, laying the columns side by side so that the byte-identical placement of c_* across conditions is visually inspectable at a glance. The wrapping role is the only thing that changes from column to column; c_* and the trailing audit stay constant, and the figure makes the single manipulated variable, the role tag, immediately legible against everything that is held fixed.

Per-model role tokens. Each open-weight model is served via Ollama with its native chat template, so the role-tag tokenization matches the model’s training-time format. Begin/end role-tag pairs are:

Qwen2.5-72B, Qwen3-30B, Phi-4-14B:

```
<|im.start|>{role}... <|im.end|>
```

Llama-3.3-70B:

```
<|start_header_id|>{role}<|end_header_id|>
... <|eot_id|>
```

Gemma-3-12B:

```
<start_of_turn>{role}... <end_of_turn>
```

gpt-oss-20B:

```
<|message_start|>{role}... <|message_end|>
```

Closed-weight models accept role strings via their JSON API surfaces (`role`: “system”/“user”/“assistant”/“tool”); the tokenization downstream of the API is not exposed, but the role-handle abstraction is preserved.

Tool-role realization. The `L_TOOL` condition is rendered with a `name=“calculator”` field so that the tool-return is bound to a named tool rather than to an anonymous role. On Ollama-served families, this materializes as a `tool` message with its template-specific begin/end pair and a `name` field; on the closed-weight APIs, it materializes as the API’s native tool-result message. The semantic intent is the same across templates: surface c_* as if a calculator had just returned it.

Audit, byte-identity, and reproducibility. The trailing audit is the V1 string of Table 8; V2–V5 are exercised only in the audit-wording robustness check of §4.4. The implementation pipeline, including hashing, condition assembly, judge calls, and trial logging, follows the open-source code released with the paper; per-condition prompts can be reproduced bit-for-bit by running the released `exp_relabel.py` (open-weight) and `exp_relabel_frontier.py` (closed-weight) drivers.

F Future Directions

The behavioral effect and its controls are established within the verifiable-task scope of this paper, and the addressability account they support opens several directions in which the same role-handle mechanism can be pushed further. A per-task analysis correlating flips with properties of c_* , its length, its distance from the audit, and the number and type of competing intermediates, would resolve the mechanism at finer grain than the cell-level lifts reported here and is a natural next use of the released trial logs. A role-randomization control that shuffles role tags across the whole conversation would further sharpen the attribution, separating the contribution of the injected handle from any global shift in role-priority cues; the byte-identity machinery already in place

makes such a control a direct extension rather than a new pipeline. On the safety side, the bare-injection regime of the adversarial mirror is the conservative end of a spectrum: pairing richer, narrative-coherent payloads with calculator availability would map how the default symmetry gives way under stronger framing, connecting our null result to the higher attack rates reported in the memory-poisoning literature and locating the threshold at which trust framing begins to dominate.

Beyond these direct controls, three longer-horizon questions follow naturally from the mechanism. First, the account predicts that the effect should transfer to any domain whose intermediates can be named as discrete objects, so extending the relabel from single-value claims to structured artifacts, a buggy diff in code debugging or a faulty step in multi-step planning, is a concrete test of the addressability hypothesis at scale. Second, since the lever is purely structural, a lightweight training signal that tags thought-internal substrings as self-addressable objects could internalize the same handle without an external role, converting the harness-level intervention into a training-level reliability property. Third, the explicit-correction gain and the small, non-significant final-answer movements of Appendix A.4 together define a design space for production harnesses: canonical handle phrases and a verify-before-accept default on `user` and `system` `<memory>` content are concrete starting policies whose trade-offs can now be measured directly. To anchor all of these as the field’s model snapshots and role-token conventions drift, we release a benchmark split of prompts with hashed c_* placements alongside the code of Appendix E, so that the effect and its controls can be re-measured on any future model under byte-identical conditions.